

R Workshop – Day 6 – Coding Statistical Models from Their Equations

Applied R for Statistical Teaching, Research and Reproducible Analysis

Contents

1	R Workshop – Day 6 – Coding Statistical Models from Their Equations	2
1.1	Learning objectives	2
1.2	Capstone link	2
1.3	Segment plan (3 hours)	2
1.4	Part 1 – Concept bridge (Core workshop activity)	2
1.4.1	Notation and dimensions	2
1.4.2	The normal equations	3
1.5	Part 2 – Guided coding (Core workshop activity)	3
1.5.1	2.1 Build the design matrix	3
1.5.2	2.2 Compute OLS coefficients directly	3
1.5.3	2.3 Fitted values and residuals	3
1.5.4	2.4 A small custom OLS function	4
1.5.5	2.5 Validate against <code>lm()</code>	4
1.6	Part 3 – Participant practice (Core workshop activity)	5
1.7	Optional demonstration	5
1.8	Reference or take-home material	6
1.9	Daily deliverable	6
1.9.1	Evidence log entry	6
1.10	Facilitator notes	6

1 R Workshop – Day 6 – Coding Statistical Models from Their Equations

Session 6 of 7 | Duration: 3 hours Programme: Applied R for Statistical Teaching, Research and Reproducible Analysis

1.1 Learning objectives

By the end of this session you will be able to:

1. Read ordinary least squares (OLS) notation and identify the dimensions of each matrix involved.
2. Build a design matrix from a formula and a data frame.
3. Derive and compute OLS coefficient estimates directly from matrix algebra.
4. Calculate fitted values and residuals from first principles.
5. Implement a small, reusable custom OLS function.
6. Validate that function against `lm()`.
7. Discuss one numerical failure mode, such as rank deficiency or poor conditioning.

1.2 Capstone link

Today you open the “black box” behind the Day 5 model: you reproduce (or partially audit) its coefficients by computing them directly from the design matrix, rather than from `lm()`.

1.3 Segment plan (3 hours)

Segment	Time	Purpose
Opening and recap	10 min	Recap the Day 5 model, preview today’s equation-to-code work
Concept bridge	25 min	OLS notation, dimensions, the normal equations
Guided coding	60 min	Design matrix, manual OLS, custom function
Participant practice	45 min	Validate against <code>lm()</code> , test one failure mode
Interpretation and reporting	25 min	Explain the equation in your own words
Evidence log and capstone update	15 min	Save the equation-based implementation

1.4 Part 1 – Concept bridge (Core workshop activity)

1.4.1 Notation and dimensions

For n observations and p predictors (including the intercept), ordinary least squares assumes:

$$y = X\beta + \varepsilon$$

where y is an $n \times 1$ vector of outcomes, X is an $n \times p$ **design matrix** (a column of 1s for the intercept, plus one column per predictor), β is a $p \times 1$ vector of coefficients, and ε is an $n \times 1$ vector of errors.

1.4.2 The normal equations

OLS chooses $\hat{\beta}$ to minimise the sum of squared residuals, $(y - X\beta)^\top(y - X\beta)$. Differentiating and setting to zero gives the **normal equations**:

$$X^\top X \hat{\beta} = X^\top y \implies \hat{\beta} = (X^\top X)^{-1} X^\top y$$

This single line is the algebraic content of every `lm()` call you have run this week.

Important implementation note: the formula above assumes X includes an intercept column and is full rank (no exact linear dependence among predictor columns). Our first custom function will assume an intercept explicitly; a model fitted without one needs different degrees-of-freedom and R-squared bookkeeping, which is out of scope today (see Reference material).

1.5 Part 2 – Guided coding (Core workshop activity)

```
lecturers <- readr::read_csv("Data/processed/anchor_dataset_clean.csv",  
  ↪ show_col_types = FALSE)  
model_data <- lecturers[stats::complete.cases(  
  lecturers[, c("post_score", "pre_score", "attendance")]  
) , ]
```

We use the simpler two-predictor model `post_score ~ pre_score + attendance` today, so the matrix algebra stays easy to follow; the same method extends to the full Day 5 model.

1.5.1 2.1 Build the design matrix

```
X <- model.matrix(~ pre_score + attendance, data = model_data)  
y <- model_data$post_score  
  
dim(X)      # n rows, 3 columns: intercept, pre_score, attendance  
head(X, 3)
```

`model.matrix()` does exactly what you would otherwise do by hand: add an intercept column of 1s and bind the predictor columns alongside it.

1.5.2 2.2 Compute OLS coefficients directly

```
XtX <- t(X) %*% X  
XtY <- t(X) %*% y  
beta_hat <- solve(XtX) %*% XtY  
beta_hat
```

Compare directly:

```
model_lm <- lm(post_score ~ pre_score + attendance, data = model_data)  
coef(model_lm)
```

The two sets of numbers should match to several decimal places.

1.5.3 2.3 Fitted values and residuals

```
fitted_manual <- X %*% beta_hat
residuals_manual <- y - fitted_manual

# Compare with lm()
max(abs(fitted_manual - fitted(model_lm)))
max(abs(residuals_manual - residuals(model_lm)))
```

1.5.4 2.4 A small custom OLS function

```
my_lm <- function(formula, data) {
  X <- model.matrix(formula, data = data)
  response_name <- all.vars(formula)[1]
  y <- data[[response_name]][stats::complete.cases(model.matrix(formula, data =
↪ data))]]

  if (colnames(X)[1] != "(Intercept)") {
    stop("my_lm() requires an intercept; interceptless formulas are not supported.")
  }

  beta_hat <- solve(t(X) %*% X) %*% (t(X) %*% y)
  fitted_values <- X %*% beta_hat
  resid_values <- y - fitted_values

  n <- nrow(X)
  p <- ncol(X)
  rss <- sum(resid_values^2)
  tss <- sum((y - mean(y))^2)
  r_squared <- 1 - rss / tss
  sigma2 <- rss / (n - p)
  var_beta <- sigma2 * solve(t(X) %*% X)
  se_beta <- sqrt(diag(var_beta))

  list(
    coefficients = setNames(as.vector(beta_hat), colnames(X)),
    std_errors = setNames(se_beta, colnames(X)),
    fitted_values = as.vector(fitted_values),
    residuals = as.vector(resid_values),
    r_squared = r_squared,
    df_residual = n - p
  )
}
```

1.5.5 2.5 Validate against lm()

```
my_fit <- my_lm(post_score ~ pre_score + attendance, data = model_data)
my_fit$coefficients
coef(model_lm)

my_fit$r_squared
summary(model_lm)$r_squared
```

```
my_fit$std_errors
summary(model_lm)$coefficients[, "Std. Error"]
```

All three comparisons should agree closely. Small differences in the last few decimal places are expected and come from differing numerical routines, not from a mistake.

1.6 Part 3 – Participant practice (Core workshop activity)

1. Run `my_lm()` on the Day 5 model formula (`post_score ~ pre_score + attendance + training_track`) and confirm coefficients, R-squared and standard errors match `lm()`.

Hint: `training_track` must be converted to numeric dummy columns – `model.matrix()` does this automatically once `training_track` is a factor.

2. Deliberately create a **rank-deficient** design matrix (e.g. include both `attendance` and an exact duplicate column `attendance_copy <- attendance`) and run it through `my_lm()`. Read and explain the resulting error from `solve()`.
3. Investigate **poor conditioning**: compute `kappa(t(X) %*% X)` for the well-specified model and for a version where one predictor is rescaled to a tiny range (e.g. `attendance / 10000`). Note how the condition number changes.
4. Write two or three sentences, in your own words, explaining what $(X^T X)^{-1} X^T y$ is doing – as if explaining it to a colleague who knows regression results but has never seen the matrix form.

A facilitator-reviewed solution is in `Solution-Scripts/day6_manual_ols_solution.R`.

1.7 Optional demonstration

- **Bootstrap uncertainty** – resample rows with replacement, refit `my_lm()` each time, and compare the bootstrap standard errors to the analytic ones above.
- **Maximum likelihood** – show that, under normal errors, the OLS estimator is also the maximum-likelihood estimator (a one-paragraph derivation, not a full implementation).
- **Detailed conditioning analysis** – `solve(t(X) %*% X)` versus `MASS::ginv()` when $X^T X$ is nearly singular.
- **Extended inference** – compute *t*-statistics and p-values from `std_errors` inside `my_lm()` and compare to `summary(model_lm)`.

```
# Optional: bootstrap standard error for the attendance coefficient
set.seed(2024)
boot_coefs <- replicate(500, {
  boot_rows <- sample(seq_len(nrow(model_data)), replace = TRUE)
  boot_fit <- my_lm(post_score ~ pre_score + attendance, data = model_data[boot_rows,
↪  ])
  boot_fit$coefficients["attendance"]
})
sd(boot_coefs)
```

1.8 Reference or take-home material

- **QR-based fitting:** `lm()` does not actually invert $X^T X$ – it uses a QR decomposition of X for better numerical stability. Explicit matrix inversion (as we did today, for teaching clarity) is more sensitive to poor conditioning than QR-based methods, and is avoided in production statistical software.
 - **Interceptless-model extensions:** if a model genuinely has no intercept ($y \sim x - 1$), R-squared, degrees of freedom and the overall F-test all require different formulas; see `?summary.lm` for how R itself handles this case.
-

1.9 Daily deliverable

An equation-based implementation (`my_lm()` and its outputs) that reproduces or audits part of the Day 5 model, with a short written explanation of the matrix algebra in your own words.

1.9.1 Evidence log entry

Complete the Day 6 row in `Workshop-Evidence-Log.md`: confirm your custom function matches `lm()` and note what the rank-deficiency exercise taught you about when a model cannot be fitted at all.

1.10 Facilitator notes

- The rank-deficiency exercise should produce a clear, interpretable error from `solve()` (a singular-matrix error); if a participant's `attendance_copy` is not an exact duplicate, the matrix may instead be merely ill-conditioned rather than singular – worth distinguishing live.
- Numerical agreement with `lm()` should be exact to at least 6-8 significant figures for the well-conditioned cases; larger discrepancies usually indicate a data-preparation mismatch (e.g. different rows being dropped due to missingness) rather than an algebra error.
- This session is mathematically the densest of the workshop. If the group needs more guided-coding time, drop the bootstrap optional demonstration first.